

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Fundada en 1551

FACULTAD DE INGENIERÍA DE SISTEMA E INFORMÁTICA
ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE - LIMA



Proyecto Grupal - Informe 01

Integrantes:

Guevara Chavez Luis Rodrigo

Núñez Cardenas Ivan Joaquin

Bejar Mallma Harian Aaron

Arancivia Salas Christian Gabriel

Rojas Rojas Max Fernando

Docente:

Juan Gamarra Moreno

Asignatura:

Inteligencia Artificial

Bloque A — Programas en Java	3
Informe 1 — Búsqueda Informada y No Informada (Java)	3
1. Definición Formal del Problema	3
2. Justificación Teórica de las Heurísticas	3
Heurística 1: Distancia Manhattan (h_1)	3
Heurística 2: Fichas Mal Ubicadas (h_2)	4
3. Diseño de la Solución	5
4. Análisis de Resultados Matemáticos y de Rendimiento	14
Ejecución con ingreso manual	14
Matriz Comparativa (Casos Estándar)	20
5. Conclusiones	24
6. Documentación de prompts	25
7. Referencias	29

Bloque A — Programas en Java

Informe 1 — Búsqueda Informada y No Informada (Java)

Objetivo: modelar un problema como búsqueda en el espacio de estados e implementar algoritmos ciegos e informados.

Problema utilizado: 8-puzzle

1. Definición Formal del Problema

El problema del 8-puzzle se clasifica como un problema de optimización combinatoria en un espacio de estados finito (Russell & Norvig, 2020). Consiste en una cuadrícula de 3 x 3 con 8 fichas numeradas del 1 al 8 y un espacio vacío (representado por el número 0).

- **Estados:** Un estado se define como la configuración específica de las fichas en la matriz. Matemáticamente, es una permutación del conjunto {0, 1, 2, 3, 4, 5, 6, 7, 8}. Para optimizar la manipulación, el estado se representa en memoria como un arreglo unidimensional de tamaño 9.
- **Estado Inicial:** Configuración arbitraria ingresada por el usuario (ej. [8, 6, 7, 2, 5, 4, 3, 0, 1]).
- **Estado Meta:** Configuración objetivo unificada: [1, 2, 3, 4, 5, 6, 7, 8, 0].
- **Operadores / Acciones:** Movimientos del espacio vacío (0) en cuatro direcciones posibles:
 - *ARRIBA:* Válido si la fila del 0 es > 0.
 - *ABAJO:* Válido si la fila del 0 es < 2.
 - *IZQUIERDA:* Válido si la columna del 0 es > 0.
 - *DERECHA:* Válido si la columna del 0 es < 2.
- **Costo de las Acciones:** Costo uniforme $c(s, a, s') = 1$ por cada movimiento realizado.

2. Justificación Teórica de las Heurísticas

Para el problema, no bastó con crear los algoritmos óptimos, también se introdujeron las heurísticas necesarias para mejorar el rendimiento de los algoritmos y que estos optimicen mejor la búsqueda de las soluciones, a continuación se detallan las heurísticas usadas

Heurística 1: Distancia Manhattan (h1)

La distancia Manhattan se calcula sumando las distancias absolutas verticales y horizontales desde la posición actual de cada ficha hasta su posición correcta en el estado meta.

$$h_1(n) = \sum_{i=1}^8 |x_{actual,i} - x_{meta,i}| + |y_{actual,i} - y_{meta,i}|$$

Donde el índice i representa el número de la ficha concreta que estamos evaluando. Como el espacio vacío (0) no se cuenta para la distancia (solo se mueven las fichas numeradas), la sumatoria va desde la ficha $i = 1$ hasta la ficha 8.

Para cualquier ficha i , necesitamos comparar dos posiciones en el tablero: **dónde está ahora** y **dónde debería estar** cuando el puzzle se resuelve.

- $x_{actual,i}$: Es la **fila actual** donde se encuentra la ficha i en el tablero que estás evaluando.
- $y_{actual,i}$: Es la **columna actual** donde se encuentra la ficha i en el tablero que estás evaluando.
- $x_{meta,i}$: Es la **fila objetivo (meta)** donde *debería* estar la ficha i en el estado resuelto.
- $y_{meta,i}$: Es la **columna objetivo (meta)** donde *debería* estar la ficha i en el estado resuelto.

Admisibilidad: Es admisible porque asume un escenario ideal donde cada ficha puede moverse directamente a su posición destino de manera lineal sin interactuar o bloquearse con otras fichas. Como el camino real siempre requerirá sortear obstáculos, la distancia Manhattan de manera estricta **nunca sobreestima** el costo real ($h(n) \leq h^*(n)$).

Consistencia: Una heurística es consistente si, para cada nodo n y cada sucesor n' generado por una acción a , se cumple que $h(n) \leq c(s, a, s') + h(n')$. Dado que mover una ficha cambia su posición en un solo paso, la distancia Manhattan se modifica exactamente en $+1$ o -1 . Por lo tanto, $\Delta h \leq 1$. Como el costo del paso $c(s, a, s') = 1$, la desigualdad triangular se mantiene formalmente.

Por que la distancia Manhattan es una buena heurística para el 8-puzzle

- Es admisible y consistente, por lo que A^* con esta heurística es completo y óptimo.
- Es más informativa que la heurística de fichas mal ubicadas: para cualquier estado, la distancia Manhattan es siempre mayor o igual al número de fichas mal ubicadas, ya que cada ficha mal ubicada aporta al menos 1 a la distancia Manhattan pero puede aportar más si está lejos de su posición meta.
- Una heurística más informada (mayor valor de h , siempre que siga siendo admisible) reduce drásticamente el número de nodos que A^* necesita expandir.

Heurística 2: Fichas Mal Ubicadas (h_2)

Esta función cuenta el número de piezas que no se encuentran en la casilla correspondiente al estado objetivo.

$$h_2(n) = \sum_{i=1}^8 1_{(actual,i \neq meta,i)}$$

El símbolo 1 representa una función lógica que actúa como un contador binario dentro de la sumatoria.

- Si la condición dentro del paréntesis es **verdadera**, la función devuelve un **1**.
- Si la condición dentro del paréntesis es **falsa**, la función devuelve un **0**.

la condición es $(actual, i \neq meta, i)$, lo que significa: "¿El número que está en la casilla i del tablero actual es **diferente** al número que debería estar en esa misma casilla en el tablero meta?".

El valor que se obtenga será un 1 cuando no haya un número en el espacio donde debería estar, y 0 cuando este en su posición correcta, al final se realiza una sumatoria, y dependiendo del valor, la heurística nos dirá el número mínimo de movimientos que se deben realizar para llegar a la solución

Estados sin solución - paridad de inversiones

Si tomamos los valores de las casillas y los colocamos en una fila, contando la cantidad de precedentes obtendremos la paridad, tomemos por ejemplo el siguiente tablero: [1, 2, 0, 3, 4, 8, 5, 7, 6]

sin contar el 0 tenemos:

8 precede al 5, 7, 6 — 3 inversiones

7 precede al 6 — 1 inversión

sumando las inversiones nos quedan 4 inversiones, como es par es resoluble

Si tuviéramos una tabla [1, 2, 3, 4, 5, 6, 8, 7, 0]

8 precede al 7 — 1 inversión

Como sólo hay 1 inversión, esta es impar y no es resoluble

Esto se debe a que el estado meta tiene inversiones 0, lo que en la práctica se toma como un número par, cada movimiento que se hace, ya sea horizontal o vertical, no hace cambios en la paridad del estado, es decir si empezamos con un estado con inversiones impares, terminaremos con un estado con inversiones impares y viceversa, por lo que es necesario identificar los estados que no son resolubles por los algoritmos

3. Diseño de la Solución

El código se organizó siguiendo la separación clásica entre la representación del problema (Estado) y la maquinaria de búsqueda (Nodo, Buscador), lo que facilita reutilizar exactamente la misma lógica de generación de sucesores y de heurísticas en los cuatro algoritmos implementados.

Estructura de clases

- **Estado:** representa el tablero (arreglo de 9 posiciones), calcula la posición del hueco, genera sucesores válidos aplicando los 4 operadores, calcula las heurísticas (Manhattan y fichas mal ubicadas) y determina si un tablero es resoluble.

```

19 public class Estado { 24 usages
36 }
37
38
39 /** Comprueba si este estado es el estado meta. */
40 public boolean esMeta() { 4 usages
41     return Arrays.equals(tablero, META);
42 }
43
44 /**
45  * OPERADORES/ACCIONES del problema: genera los estados sucesores validos
46  * moviendo el espacio vacio ARRIBA, ABAJO, IZQUIERDA o DERECHA, siempre
47  * que el movimiento no salga del tablero. Devuelve pares (operador, estado).
48  */
49 public List<Sucesor> generarSucesores() { 1 usage
50     List<Sucesor> sucesores = new ArrayList<>(initialCapacity: 4);
51     int fila = posVacio / N;
52     int col = posVacio % N;
53
54     // {deltaFila, deltaColumna, codigoOperador}
55     int[][] movimientos = {
56         {-1, 0, 0}, // ARRIBA (el hueco sube; la ficha de arriba baja)
57         {1, 0, 1}, // ABAJO
58         {0, -1, 2}, // IZQUIERDA
59         {0, 1, 3} // DERECHA
60     };
61     String[] nombres = {"ARRIBA", "ABAJO", "IZQUIERDA", "DERECHA"};
62
63     for (int[] mov : movimientos) {
64         int nf = fila + mov[0];
65         int nc = col + mov[1];
66         if (nf < 0 || nf >= N || nc < 0 || nc >= N) continue; // fuera del tablero: invalido

```

```

19 public class Estado { 24 usages
121     public String toStringTablero() { 3 usages
128         sb.append("\n");
129     }
130     return sb.toString();
131 }
132
133 /**
134  * Determina si un tablero es resoluble usando la regla de paridad de
135  * inversiones: para un tablero 3x3, el estado es resoluble si y solo si
136  * el numero de inversiones (pares fuera de orden, ignorando el 0) es PAR.
137  */
138 @ public static boolean esResoluble(int[] tablero) { 1 usage
139     int[] sinCero = new int[8];
140     int idx = 0;
141     for (int v : tablero) if (v != 0) sinCero[idx++] = v;
142     int inversiones = 0;
143     for (int i = 0; i < sinCero.length; i++)
144         for (int j = i + 1; j < sinCero.length; j++)
145             if (sinCero[i] > sinCero[j]) inversiones++;
146     return inversiones % 2 == 0;
147 }
148
149 /** Par (operador aplicado, estado resultante), usado al expandir un Estado. */
150 public static class Sucesor { 4 usages
151     public final String operador; 2 usages
152     public final Estado estado; 2 usages
153     public Sucesor(String operador, Estado estado) { 1 usage
154         this.operador = operador;
155         this.estado = estado;
156     }
157 }

```

En esta clase se verifica la paridad del estado con el método es Resoluble, que usando los valores del tablero con dos vectores, uno una posición delante del otro,, sin contar el 0, calcula el número de inversiones que existe en el tablero y devuelve un valor booleano true si es par y false si es impar.

- **Nodo:** envuelve un Estado dentro del árbol de búsqueda; agrega el puntero al nodo padre, el operador que se aplicó para llegar a él, el costo acumulado $g(n)$, el valor heurístico $h(n)$ y expone $f(n) = g(n) + h(n)$.

```

public class Nodo implements Comparable<Nodo> { 34 usages
    public int getG() { return g; } 8 usages
    public int getH() { return h; } 1 usage
    public void setH(int h) { this.h = h; } 4 usages
    public int getF() { return g + h; } 2 usages

    /** Expande este nodo generando sus nodos hijos (uno por cada sucesor valido del estado). */
    public java.util.List<Nodo> expandir() { 4 usages
        java.util.List<Nodo> hijos = new java.util.ArrayList<>();
        for (Estado.Sucesor s : estado.generarSucesores()) {
            hijos.add(new Nodo(s.estado, padre: this, s.operador, g: this.g + 1));
        }
        return hijos;
    }

    /** Reconstruye la secuencia de operadores desde la raiz hasta este nodo. */
    public java.util.List<String> reconstruirCamino() { 1 usage
        java.util.LinkedList<String> camino = new java.util.LinkedList<>();
        Nodo actual = this;
        while (actual.padre != null) {
            camino.addFirst(actual.operador);
            actual = actual.padre;
        }
        return camino;
    }

    /** Reconstruye la secuencia de estados (tableros) desde la raiz hasta este nodo. */
    public java.util.List<Estado> reconstruirEstados() { 1 usage
        java.util.LinkedList<Estado> camino = new java.util.LinkedList<>();
        Nodo actual = this;
        while (actual != null) {
            camino.addFirst(actual.estado);
        }
    }
}

```

- **Buscador (interfaz):** define el contrato común `buscar(Estado inicial): ResultadoBusqueda`, implementado por las cuatro clases concretas.

```

/**
 * Interfaz comun para todos los algoritmos de busqueda (BFS, DFS, Greedy, A*).
 * Cada clase concreta implementa buscar(), que recibe el estado inicial y
 * devuelve un ResultadoBusqueda con la solucion (si existe) y las metricas
 * de desempeño solicitadas (nodos expandidos, longitud de la solucion,
 * tiempo de ejecucion, etc.).
 */
public interface Buscador { 10 usages 4 implementations

    ResultadoBusqueda buscar(Estado inicial); 3 usages 4 implementations

    /** Nombre descriptivo del algoritmo (para reportes/tablas comparativas). */
    String getNombre(); 9 usages 4 implementations
}

```


- **BFS, DFS, Greedy, A*:** implementaciones concretas de Buscador, cada una con su propia estructura de datos para la frontera (cola FIFO, pila LIFO o cola de prioridad) y su propio criterio de expansión.
- **ResultadoBusqueda:** objeto de valor que agrupa las métricas de una ejecución (nodo solución, nodos expandidos, nodos generados, tiempo en milisegundos,

```

1  /**
2   * Encapsula el resultado de ejecutar un algoritmo de busqueda sobre una
3   * instancia del 8-puzzle, junto con las metricas de comparacion pedidas:
4   * - nodos expandidos (estados generados y evaluados)
5   * - longitud/costo de la solucion (numero de movimientos)
6   * - tiempo de ejecucion en milisegundos
7   * - si se encontro solucion (completitud practica en esta ejecucion)
8   */
9  public class ResultadoBusqueda { 16 usages
10
11     public final String algoritmo; 3 usages
12     public final boolean solucionEncontrada; 4 usages
13     public final Nodo nodoSolucion; // null si no se encontro solucion 4 usages
14     public final int nodosExpandidos; 4 usages
15     public final int nodosGenerados; 4 usages
16     public final long tiempoMs; 4 usages
17     public final int profundidadMaximaAlcanzada; 2 usages
18
19     public ResultadoBusqueda(String algoritmo, boolean solucionEncontrada, Nodo nodoSolucion, 8 usages
20         int nodosExpandidos, int nodosGenerados, long tiempoMs,
21         int profundidadMaximaAlcanzada) {
22         this.algoritmo = algoritmo;
23         this.solucionEncontrada = solucionEncontrada;
24         this.nodoSolucion = nodoSolucion;
25         this.nodosExpandidos = nodosExpandidos;
26         this.nodosGenerados = nodosGenerados;
27         this.tiempoMs = tiempoMs;
28         this.profundidadMaximaAlcanzada = profundidadMaximaAlcanzada;
29     }
30
31     > public int getLongitudSolucion() { return nodoSolucion == null ? -1 : nodoSolucion.getG(); }
34

```

profundidad máxima alcanzada).

- **Main:** interfaz de línea de comandos; permite ingresar el estado inicial manualmente, elegir uno predefinido, seleccionar el algoritmo a ejecutar, visualizar la solución paso a paso y correr automáticamente la tabla comparativa de los 3 casos de prueba.

Nota: si en el ingreso manual, se ingresó un estado irresoluble se preguntara al usuario si quiere desarrollarlo de todas formas, aunque todos los algoritmos arrojaran “Sin Solución” en la tabla de resultados.

```
5 public class Main {
6
7     // Casos de prueba predefinidos (facil, medio, dificil)
8     static final int[] FACIL = {1, 2, 3, 4, 5, 6, 0, 7, 8}; // 2 movimientos de la meta 3 usages
9     static final int[] MEDIO = {1, 2, 3, 0, 4, 6, 7, 5, 8}; // dificultad intermedia 3 usages
10    static final int[] DIFICIL = {8, 6, 7, 2, 5, 4, 3, 0, 1}; // caso clasico "dificil" (31 movimientos optimos
11
12    public static void main(String[] args) {
13        Scanner sc = new Scanner(System.in);
14
15        System.out.println("=====");
16        System.out.println("8-PUZZLE - Comparacion de Algoritmos de Búsqueda");
17        System.out.println("Inteligencia Artificial");
18        System.out.println("=====");
19
20        while (true) {
21            System.out.println("\nMENU PRINCIPAL");
22            System.out.println("1. Ingresar estado inicial manualmente");
23            System.out.println("2. Usar estado predefinido (facil / medio / dificil)");
24            System.out.println("3. Ejecutar tabla comparativa (3 casos de prueba, los 4 algoritmos)");
25            System.out.println("4. Salir");
26            System.out.print("Seleccione una opcion: ");
27            String opcion = sc.nextLine().trim();
28
29            switch (opcion) {
30                case "1": {
31                    int[] tablero = leerTableroManual(sc);
32                    if (tablero != null) ejecutarMenuAlgoritmos(sc, tablero);
33                    break;
34                }
35                case "2": {
36                    int[] tablero = elegirPredefinido(sc);
```

Manejo de estados repetidos

Dado que las acciones del 8-puzzle son reversibles (mover el hueco hacia arriba y luego hacia abajo regresa al mismo estado), el espacio de estados contiene ciclos. Los cuatro algoritmos implementados usan estructuras de la colección estándar de Java para evitar reexpandir estados ya visitados:

- BFS y Greedy: un HashSet de estados visitados, donde la clave es la representación textual del tablero.
- DFS: un HashSet global de visitados (convirtiéndolo en un DFS de grafo) además del límite de profundidad, para controlar tanto los ciclos como las ramas excesivamente largas.
- A*: un mapa HashMap que registra el mejor costo $g(n)$ conocido por estado, permitiendo reinsertar un estado en la frontera si se descubre un camino más barato hacia el (requisito de corrección para A* como búsqueda de grafo), junto con un conjunto de nodos ya cerrados

Mecanismo de algoritmos implementados

A. Búsqueda en Anchura (BFS)

- **Estructura Empleada:** Cola FIFO (LinkedList).
- **Mecanismo:** Expande sistemáticamente todos los nodos de un nivel de profundidad antes de pasar al siguiente. Los nuevos sucesores descubiertos se encolan al final.

- **Impacto:** Garantiza hallar el camino óptimo en costos uniformes, pero sufre de una alta penalización espacial al almacenar en memoria fronteras masivas de nodos antes de llegar a la meta.

```

19  public class BFS implements Buscador { 3 usages
25  public ResultadoBusqueda buscar(Estado inicial) {
27
28      Queue<Nodo> frontera = new LinkedList<>();
29      Set<String> visitados = new HashSet<>(); // control de estados repetidos
30
31      Nodo raiz = new Nodo(inicial, padre: null, operador: null, g: 0);
32      frontera.add(raiz);
33      visitados.add(inicial.clave());
34
35      int nodosExpandidos = 0;
36      int nodosGenerados = 1;
37      int profundidadMax = 0;
38
39      while (!frontera.isEmpty()) {
40          Nodo actual = frontera.poll(); // extrae el mas antiguo (FIFO)
41
42          if (actual.getEstado().esMeta()) {
43              long t1 = System.nanoTime();
44              return new ResultadoBusqueda(getNombre(), solucionEncontrada: true, actual, nodosExpandidos,
45                  nodosGenerados, tiempoMs: (t1 - t0) / 1_000_000, profundidadMax);
46          }
47
48          nodosExpandidos++;
49          profundidadMax = Math.max(profundidadMax, actual.getG());
50
51          for (Nodo hijo : actual.expandir()) {
52              String clave = hijo.getEstado().clave();
53              if (!visitados.contains(clave)) {
54                  visitados.add(clave);
55                  frontera.add(hijo);
56                  nodosGenerados++;

```

B. Búsqueda en Profundidad (DFS)

- **Estructura Empleada:** Pila LIFO (Stack).
- **Mecanismo:** Explora de forma agresiva la rama más profunda disponible. Los sucesores se apilan al frente para ser evaluados de inmediato en la siguiente iteración.
- **Mitigación de Riesgos:** Para evitar que la pila se sature de forma indefinida en ciclos infinitos, el diseño incorpora un **límite estricto de profundidad** ($L = 30$). Si un nodo alcanza $g == 30$, el algoritmo se ve obligado a realizar un *backtracking* (retroceso) ignorando los hijos de ese estado.

```

19 public class DFS implements Buscador { 3 usages
31 public ResultadoBusqueda buscar(Estado inicial) {
33
34     Deque<Nodo> pila = new ArrayDeque<>(); // pila (push/pop) = LIFO
35     Set<String> enPila = new HashSet<>(); // estados presentes en la rama actual (evita ciclos)
36
37     Nodo raiz = new Nodo(inicial, padre: null, operador: null, g: 0);
38     pila.push(raiz);
39
40     int nodosExpandidos = 0;
41     int nodosGenerados = 1;
42     int profundidadMax = 0;
43
44     // Para controlar ciclos en DFS se usa el conjunto de visitados global;
45     // esto convierte la búsqueda en un DFS de grafo (mas eficiente en la practica).
46     Set<String> visitadosGlobal = new HashSet<>();
47     visitadosGlobal.add(inicial.clave());
48
49     while (!pila.isEmpty()) {
50         Nodo actual = pila.pop(); // extrae el mas reciente (LIFO)
51
52         if (actual.getEstado().esMeta()) {
53             long t1 = System.nanoTime();
54             return new ResultadoBusqueda(getNombre(), solucionEncontrada: true, actual, nodosExpandidos,
55                 nodosGenerados, tiempoMs: (t1 - t0) / 1_000_000, profundidadMax);
56         }
57
58         profundidadMax = Math.max(profundidadMax, actual.getG());
59
60         if (actual.getG() >= profundidadMaxima) {
61             continue; // corte por profundidad: no se expande mas esta rama
62         }
63     }
64 }

```

C. Greedy Best-First Search (Búsqueda Voraz)

- **Estructura Empleada:** Cola de Prioridad (PriorityQueue).
- **Mecanismo:** Utiliza una función de evaluación que depende exclusivamente de la heurística: $f(n) = h(n)$ (distancia Manhattan). El comparador de la cola ordena los nodos situando siempre al inicio aquel elemento que estime estar geométricamente "más cerca" de la meta.
- **Impacto:** Es extremadamente rápido porque avanza en línea recta hacia lo que parece ser la solución, sacrificando por completo la noción de optimalidad histórica ($g(n)$).

```

20 public class Greedy implements Buscador { 3 usages
36 public ResultadoBusqueda buscar(Estado inicial) {
37     long t0 = System.nanoTime();
38
39     // Orden solo por h(n): la esencia de "voraz" (greedy)
40     PriorityQueue<Nodo> frontera = new PriorityQueue<>(Comparator.comparingInt(Nodo::getH));
41     Set<String> visitados = new HashSet<>();
42
43     Nodo raiz = new Nodo(inicial, padre: null, operador: null, g: 0);
44     raiz.setH(heuristica(inicial));
45     frontera.add(raiz);
46     visitados.add(inicial.clave());
47
48     int nodosExpandidos = 0;
49     int nodosGenerados = 1;
50     int profundidadMax = 0;
51
52     while (!frontera.isEmpty()) {
53         Nodo actual = frontera.poll();
54
55         if (actual.getEstado().esMeta()) {
56             long t1 = System.nanoTime();
57             return new ResultadoBusqueda(getNombre(), solucionEncontrada: true, actual, nodosExpandidos,
58                 nodosGenerados, tiempoMs: (t1 - t0) / 1_000_000, profundidadMax);
59         }
60
61         nodosExpandidos++;
62         profundidadMax = Math.max(profundidadMax, actual.getG());
63
64         for (Nodo hijo : actual.expandir()) {
65             String clave = hijo.getEstado().clave();
66             if (!visitados.contains(clave)) {

```

D. Algoritmo A-Estrella

- **Estructura Empleada:** Cola de Prioridad (PriorityQueue) combinada con un mapa de costos acumulados (HashMap<Integer, Integer>).
- **Mecanismo:** Evalúa los nodos basándose en la función matemática balanceada:

$$f(n) = g(n) + h(n)$$

La cola de prioridad reordena dinámicamente los nodos de menor a mayor valor de $f(n)$.
- **Poda y Optimización:** Al extraer un nodo de la cola, el algoritmo verifica mediante un mapa (costSoFar) si ya se conocía una ruta previa hacia esa misma configuración. Si el camino actual posee un costo $g(n)$ menor que el registrado anteriormente, el mapa se actualiza y el nodo se re-evalúa. Esta propiedad, combinada con la admisibilidad de las heurísticas utilizadas (Manhattan y Fichas mal ubicadas), es lo que garantiza matemáticamente que la primera solución extraída de la frontera sea, sin excepción, la de **costo mínimo absoluto**.

```

24 public class AEstrella implements Buscador { 6 usages
37 }
38
39 @Override 3 usages
40 public ResultadoBusqueda buscar(Estado inicial) {
41     long t0 = System.nanoTime();
42
43     PriorityQueue<Nodo> frontera = new PriorityQueue<>(); // orden natural: f(n)=g(n)+h(n)
44     Map<String, Integer> mejorCosto = new HashMap<>(); // mejor g(n) conocido por estado
45     java.util.Set<String> cerrados = new java.util.HashSet<>(); // estados ya expandidos definitivamente
46
47     Nodo raiz = new Nodo(inicial, padre: null, operador: null, g: 0);
48     raiz.setH(heuristica(inicial));
49     frontera.add(raiz);
50     mejorCosto.put(inicial.clave(), 0);
51
52     int nodosExpandidos = 0;
53     int nodosGenerados = 1;
54     int profundidadMax = 0;
55
56     while (!frontera.isEmpty()) {
57         Nodo actual = frontera.poll();
58         String claveActual = actual.getEstado().clave();
59
60         // Si ya fue cerrado con un costo igual o mejor, se descarta (entrada obsoleta).
61         if (cerrados.contains(claveActual)) continue;
62
63         if (actual.getEstado().esMeta()) {
64             long t1 = System.nanoTime();
65             return new ResultadoBusqueda(getNombre(), solucionEncontrada: true, actual, nodosExpandidos,
66                 nodosGenerados, tiempoMs: (t1 - t0) / 1_000_000, profundidadMax);
67         }

```

4. Análisis de Resultados Matemáticos y de Rendimiento

Ejecución con ingreso manual

- Caso analizado: [1, 7, 8, 2, 0, 3, 4, 5, 6]
- Algoritmo usado: BFS

MENU PRINCIPAL

1. Ingresar estado inicial manualmente
2. Usar estado predefinido (facil / medio / dificil)
3. Ejecutar tabla comparativa (3 casos de prueba, los 4 algoritmos)
4. Salir

Seleccione una opcion: 1

Ingrese los 9 valores del tablero separados por espacio (0 = espacio vacio).

Ejemplo: 1 2 3 4 0 5 6 7 8

> 1 7 8 2 0 3 4 5 6

Estado inicial:

1 7 8

2 _ 3

4 5 6

Seleccione el algoritmo:

1. BFS
2. DFS (limite de profundidad 30)
3. Greedy Best-First (heuristica Manhattan)
4. A* (heuristica Manhattan)
5. A* (heuristica fichas mal ubicadas)
6. Ejecutar TODOS y comparar

>

Seleccione el algoritmo:

1. BFS
2. DFS (limite de profundidad 30)
3. Greedy Best-First (heurística Manhattan)
4. A* (heurística Manhattan)
5. A* (heurística fichas mal ubicadas)
6. Ejecutar TODOS y comparar

> 1

Resultado: BFS | pasos=20 | expandidos=48264 generados=67028 tiempo=100 ms | prof.max=20

¿Mostrar solución paso a paso? (s/n): s

--- Paso 0 (estado inicial) ---

```
1 7 8
2 _ 3
4 5 6
```

--- Paso 1: mover ARRIBA ---

```
1 _ 8
2 7 3
4 5 6
```

--- Paso 2: mover DERECHA ---

```
1 8 _
2 7 3
4 5 6
```


--- Paso 17: mover IZQUIERDA ---

1 _ 3

4 2 6

7 5 8

--- Paso 18: mover ABAJO ---

1 2 3

4 _ 6

7 5 8

--- Paso 19: mover ABAJO ---

1 2 3

4 5 6

7 _ 8

--- Paso 20: mover DERECHA ---

1 2 3

4 5 6

7 8 _

- Algoritmo usado: DFS

Seleccione el algoritmo:

1. BFS
2. DFS (límite de profundidad 30)
3. Greedy Best-First (heurística Manhattan)
4. A* (heurística Manhattan)
5. A* (heurística fichas mal ubicadas)
6. Ejecutar TODOS y comparar

> 2

Resultado: DFS | pasos=30 | expandidos=32465 generados=53086 tiempo=61 ms | prof.max=30
¿Mostrar solución paso a paso? (s/n): s

--- Paso 0 (estado inicial) ---

```
1 7 8
2 _ 3
4 5 6
```

--- Paso 1: mover ARRIBA ---

```
1 _ 8
2 7 3
4 5 6
```

--- Paso 2: mover DERECHA ---

```
1 8 _
2 7 3
4 5 6
```

--- Paso 26: mover DERECHA ---

1 2 _

4 6 3

7 5 8

--- Paso 27: mover ABAJO ---

1 2 3

4 6 _

7 5 8

--- Paso 28: mover IZQUIERDA ---

1 2 3

4 _ 6

7 5 8

--- Paso 29: mover ABAJO ---

1 2 3

4 5 6

7 _ 8

--- Paso 30: mover DERECHA ---

1 2 3

4 5 6

7 8 _

Algoritmo usado: Todos los algoritmos

```

Seleccione el algoritmo:
1. BFS
2. DFS (límite de profundidad 30)
3. Greedy Best-First (heurística Manhattan)
4. A* (heurística Manhattan)
5. A* (heurística fichas mal ubicadas)
6. Ejecutar TODOS y comparar
> 6
BFS          | pasos=20   | expandidos=48264   generados=67028   tiempo=100   ms | prof.max=20
DFS          | pasos=30   | expandidos=32465   generados=53086   tiempo=135   ms | prof.max=30
Greedy(manhattan) | pasos=40   | expandidos=99      generados=162     tiempo=0     ms | prof.max=39
A*(manhattan) | pasos=20   | expandidos=482     generados=782     tiempo=1     ms | prof.max=19
A*(malubicadas) | pasos=20   | expandidos=3778    generados=6019    tiempo=8     ms | prof.max=19

```

Caso sin solución: [1, 7, 5, 2, 0, 3, 4, 8, 6]

```

Seleccione una opción: 1
Ingrese los 9 valores del tablero separados por espacio (0 = espacio vacío).
Ejemplo: 1 2 3 4 0 5 6 7 8
> 1 7 5 2 0 3 4 8 6
ADVERTENCIA: este estado NO tiene solución (paridad de inversiones impar).
¿Continuar de todas formas? (s/n)
s

Estado inicial:
1 7 5
2 _ 3
4 8 6

Seleccione el algoritmo:
1. BFS
2. DFS (límite de profundidad 30)
3. Greedy Best-First (heurística Manhattan)
4. A* (heurística Manhattan)
5. A* (heurística fichas mal ubicadas)
6. Ejecutar TODOS y comparar
> 6
BFS          | SIN SOLUCION | expandidos=181440   generados=181440   tiempo=539ms
DFS          | SIN SOLUCION | expandidos=35649    generados=57964    tiempo=73ms
Greedy(manhattan) | SIN SOLUCION | expandidos=181440   generados=181440   tiempo=438ms
A*(manhattan) | SIN SOLUCION | expandidos=181440   generados=190446    tiempo=626ms
A*(malubicadas) | SIN SOLUCION | expandidos=181440   generados=181920    tiempo=626ms

```

Matriz Comparativa (Casos Estándar)

Se definieron tres casos de prueba de dificultad creciente para comparar los cinco buscadores (BFS, DFS, Greedy-Manhattan, A*-Manhattan, A*-fichas mal ubicadas):

- FACIL: [1, 2, 3, 4, 5, 6, 0, 7, 8] - solución óptima de 2 movimientos.

- MEDIO: [1, 2, 3, 0, 4, 6, 7, 5, 8] - solución óptima de 3 movimientos.
- DIFÍCIL: [8, 6, 7, 2, 5, 4, 3, 0, 1] - caso clásico de referencia en la literatura, con solución óptima de 31 movimientos.

MENU PRINCIPAL

1. Ingresar estado inicial manualmente
2. Usar estado predefinido (facil / medio / dificil)
3. Ejecutar tabla comparativa (3 casos de prueba, los 4 algoritmos)
4. Salir

Seleccione una opcion: 2

1. Facil ([1, 2, 3, 4, 5, 6, 0, 7, 8])
2. Medio ([1, 2, 3, 0, 4, 6, 7, 5, 8])
3. Dificil ([8, 6, 7, 2, 5, 4, 3, 0, 1])

Seleccione: 2

Estado inicial:

```
1 2 3
_ 4 6
7 5 8
```

Seleccione el algoritmo:

1. BFS
2. DFS (límite de profundidad 30)
3. Greedy Best-First (heurística Manhattan)
4. A* (heurística Manhattan)
5. A* (heurística fichas mal ubicadas)
6. Ejecutar TODOS y comparar

> 2

Resultado: DFS | pasos=3 | expandidos=12423 generados=19376 tiempo=57 ms | prof.max=30

¿Mostrar solución paso a paso? (s/n): |

--- Paso 1: mover DERECHA ---

1 2 3

4 _ 6

7 5 8

--- Paso 2: mover ABAJO ---

1 2 3

4 5 6

7 _ 8

--- Paso 3: mover DERECHA ---

1 2 3

4 5 6

7 8 _

Para cada combinacion (caso, algoritmo) se registraron las métricas solicitadas: número de nodos expandidos, número de nodos generados, longitud/costo de la solución encontrada y tiempo de ejecución en milisegundos, medido con `System.nanoTime()` alrededor de la ejecución completa del algoritmo.

Tabla comparativa con los 3 casos de prueba y todos los algoritmos

3. Ejecutar tabla comparativa (3 casos de prueba, los 4 algoritmos)

4. Salir

Seleccione una opcion: 3

Caso	Algoritmo	Pasos	Expandidos	Generados	Tiempo(ms)
FACIL	BFS	2	6	14	0
FACIL	DFS	2	2	5	0
FACIL	Greedy(manhattan)	2	2	5	12
FACIL	A*(manhattan)	2	2	5	0
FACIL	A*(malubicadas)	2	2	5	0
MEDIO	BFS	3	16	30	0
MEDIO	DFS	3	12423	19376	29
MEDIO	Greedy(manhattan)	3	3	9	0
MEDIO	A*(manhattan)	3	3	9	0
MEDIO	A*(malubicadas)	3	3	9	0
DIFICIL	BFS	31	181438	181440	386
DIFICIL	DFS	N/A	15707	24249	19
DIFICIL	Greedy(manhattan)	51	89	145	1
DIFICIL	A*(manhattan)	31	8085	12389	38
DIFICIL	A*(malubicadas)	31	124783	148264	393

Caso	Algoritmo	Pasos	Nodos Expandidos	Nodos Generados	Tiempo (ms)
FÁCIL	BFS	2	6	14	0
FÁCIL	DFS (lim=30)	2	2	5	12
FÁCIL	greedy (Manhattan)	2	2	5	0
FÁCIL	A* (Manhattan)	2	2	5	0
FÁCIL	A* (mal ubicadas)	2	2	5	0
MEDIO	BFS	3	16	30	0
MEDIO	DFS (lim=30)	3	12423	19376	29
MEDIO	greedy (Manhattan)	3	3	9	0
MEDIO	A* (Manhattan)	3	3	9	0
MEDIO	A* (mal ubicadas)	3	3	9	0
DIFÍCIL	BFS	31	181438	181440	386
DIFÍCIL	DFS (lim=30)	SIN SOLUCIÓN	15707	24249	19
DIFÍCIL	greedy (Manhattan)	51	89	145	1
DIFÍCIL	A* (Manhattan)	31	8085	12389	38
DIFÍCIL	A* (mal ubicadas)	31	124783	148264	393

Por que A* es el más óptimo en la práctica

A* combina lo mejor de BFS y de Greedy: al usar $f(n) = g(n) + h(n)$, no abandona la garantía de optimalidad de BFS (gracias a que la distancia Manhattan es admisible y consistente), pero dirige la búsqueda usando la información heurística, evitando la explosión combinatoria de BFS. En la instancia difícil, A* con Manhattan expandió 8,085 nodos frente a los 181,438 de BFS, una reducción de más del 95%, manteniendo la misma solución óptima de 31 movimientos.

Comparado a Greedy, este algoritmo puede ser más rápido en velocidad y expande menos nodos, pero no significa que sea más óptimo, al ser un algoritmo miope (ya que no considera el costo ya invertido $g(n)$, solo sigue el camino guiado por $h(n)$) este necesita muchos más pasos para llegar a la solución, pero lo hace en menos tiempo al considerar menos opciones. Por otro lado, A* usando h_1 si considera que el camino a buscar sea el más corto posible, por lo que antes de arriesgarse a seguir un camino a ciegas, expanda los nodos y explora las rutas alternativas que tiene a disposición, lo que se ve reflejado en el número de pasos (31 pasos) significativamente menor a Greedy, lo que en el papel es lo mas optimo.

Por último, la elección de la heurística es determinante: A* con la heurística de fichas mal ubicadas, siendo también admisible y encontrando la solución óptima, expande 124,783 nodos en la misma instancia, un orden de magnitud más que con Manhattan. Esto confirma en la práctica el principio teórico de dominancia entre heurísticas admisibles, cuanto más informada (mayor valor, sin dejar de ser admisible) es la heurística, menos nodos necesita expandir A* para garantizar la misma solución óptima.

Limitaciones observadas de DFS

DFS mostró dos problemas característicos: (1) en la instancia MEDIO, aunque encontró la solución óptima, extendió 12,423 nodos frente a los 16 de BFS o los 3 de A*, porque profundiza sin ningún criterio de cercanía a la meta y puede recorrer ramas irrelevantes antes de tropezar con la solución; (2) en la instancia DIFÍCIL, DFS no encontró ninguna solución, ya que la solución óptima (31 movimientos) supera el límite de profundidad configurado (30). Esto ilustra la tensión inherente al límite de profundidad: un límite bajo compromete la completitud, mientras que un límite alto (o sin límite) puede llevar a recorrer caminos arbitrariamente largos en un espacio de estados con ciclos.

Limitaciones observadas de BFS

BFS garantiza siempre la solución óptima, pero paga un costo alto en nodos generados y expandidos a medida que crece la profundidad de la solución: en la instancia DIFÍCIL debió expandir 181,438 nodos (prácticamente el espacio de estados completo del 8-puzzle, que tiene 181,440 configuraciones resolubles) para encontrar la solución, lo que se traduce en un tiempo de ejecución notablemente mayor que el de A*. Esta es la explosión combinatoria característica de la búsqueda ciega en espacios de estados con factor de ramificación mayor que 1.

5. Conclusiones

- Los cuatro algoritmos implementados cumplen su especificación teórica: BFS y A* (con heurística admisible) son completos y óptimos; DFS con límite de profundidad puede fallar si la solución excede el límite; Greedy es rápido pero no garantiza optimalidad.
- La distancia Manhattan es una heurística admisible y consistente para el 8-puzzle, lo que garantiza formalmente que A* con esta heurística encuentra siempre la solución de costo mínimo.
- La elección de la heurística tiene un impacto muy significativo en el rendimiento de A*. En el caso más difícil probado, usar distancia Manhattan en lugar de fichas mal ubicadas redujo los nodos expandidos en más de un 90%, sin sacrificar la optimalidad de la solución.
- A* Resultó ser, en la práctica, el algoritmo con mejor relación entre calidad de la solución (siempre óptima) y costo computacional (nodos expandidos y tiempo), confirmando por qué es el algoritmo de referencia para este tipo de problemas de búsqueda con heurísticas admisibles.

6. Documentación de prompts

N°	P01
Objetivo del prompt	Obtener la definición formal del problema del 8-puzzle y la estructura base del código en Java, incluyendo representación de estados, operadores y verificación de solubilidad.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Genera clases de código para implementar el problema del 8-puzzle en Java. Por favor, proporciona: 1. La definición formal del problema (estados, estado inicial, estado meta, operadores/acciones, costos). 2. La estructura de una clase Estado que represente el tablero 3x3 usando un arreglo unidimensional de 9 posiciones. 3. Métodos para: generar movimientos válidos (ARRIBA, ABAJO, IZQUIERDA, DERECHA), calcular la distancia Manhattan, y verificar si el puzzle es soluble (usando inversiones). 4. Un método main que permita ingresar un estado inicial por teclado y muestre el estado en formato de tablero 3x3. El código debe estar en Java, bien comentado y con buenas prácticas de programación".
Resultado / cómo se usó	Se obtuvo una clase Estado completa con constructor, métodos para movimientos, cálculo de Manhattan y verificación de solubilidad. Se incorporó como base para todos los algoritmos de búsqueda posteriores. La verificación de solubilidad permitió descartar estados no resolubles antes de ejecutar cualquier algoritmo.

Validación / ajuste del equipo	Se verificó que el cálculo de inversiones fuera correcto (incluyendo el espacio vacío). Se ajustó el método equals() y hashCode() para usar correctamente en colecciones. Se añadió un método clone() para facilitar la exploración de estados. Se probó con estados conocidos (fáciles y difíciles) para validar la generación de movimientos.
--------------------------------	---

N°	P02
Objetivo del prompt	Implementar los algoritmos de búsqueda no informados (BFS y DFS) para el 8-puzzle, con manejo de estados visitados, control de profundidad y generación de estadísticas de rendimiento.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Continúa la implementación del 8-puzzle en Java. Ahora necesito implementar algoritmos de búsqueda, considera lo siguiente: 1. BFS (Breadth-First Search) usando una cola FIFO (LinkedList). 2. DFS (Depth-First Search) usando una pila LIFO (Stack) con límite de profundidad de 30 para evitar bucles infinitos. 3. Ambos algoritmos deben: - Usar un HashSet para estados visitados. - Mantener el nodo padre para reconstruir la solución. - Retornar: nodos expandidos, costo de la solución, tiempo de ejecución y camino de movimientos. 4. Una clase principal que permita seleccionar el algoritmo y ejecutarlo con el estado inicial definido previamente. Incluye comentarios explicativos y manejo de casos donde no se encuentra solución."
Resultado / cómo se usó	Se obtuvieron las clases BFS y DFS con toda la lógica de búsqueda. Ambas implementaciones generaban estadísticas detalladas (nodos expandidos, profundidad, tiempo). Se añadió la reconstrucción del camino de solución a partir de los nodos padre.
Validación / ajuste del equipo	Se probaron ambos algoritmos con estados de complejidad variable. Se detectó que DFS sin límite de profundidad podía circular, por lo que se reforzó el límite según el mejor caso teórico de movimientos (30 movimientos maximo) y se implementó un control de nodos visitados en la misma ruta. Se ajustó la estructura de Nodo para incluir el operador que generó el estado. Se validó que BFS siempre encuentra la solución óptima (menor número de movimientos).

N°	P03
Objetivo del prompt	Implementar la estructura base de los algoritmos informados (Greedy Best-First y A*) con sus respectivas colas de prioridad y el sistema de evaluación heurística.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para el 8-puzzle en Java, necesito implementar la base de los algoritmos informados, toma en cuenta lo siguiente antes de implementar: 1. Una interfaz o clase abstracta Buscador que defina el método buscar(Estado inicial). 2. La clase Nodo que contenga: estado, nodo padre, operador, profundidad (g), costo acumulado, y heurística (h). 3. Greedy Best-First Search con PriorityQueue ordenada por h(n) (distancia Manhattan). 4. A con PriorityQueue ordenada por $f(n) = g(n) + h(n)$. 5. Mecanismo para evitar estados repetidos (HashSet de estados visitados). 6. Método para reconstruir el camino solución desde el nodo meta.."
Resultado / cómo se usó	Se obtuvieron las estructuras base de Nodo, Buscador, Greedy y A*. La implementación de PriorityQueue con Comparador permitió ordenar eficientemente los nodos según la heurística. El manejo de estados visitados evitó ciclos y mejoró el rendimiento.
Validación / ajuste del equipo	Se verificó que los comparadores funcionaran correctamente en PriorityQueue. Se ajustó la clase Nodo para implementar CompareTo y facilitar la ordenación en A*. Se probó que ambos algoritmos encontrarán solución para estados de complejidad media.

N°	P04
Objetivo del prompt	Completar la implementación de Greedy y A*, añadir heurísticas alternativas (fichas mal ubicadas), y crear un sistema de selección de heurística.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Completa la implementación de los algoritmos informados para el 8-puzzle: 1. Finalizar Greedy Best-First Search con la lógica completa de expansión de nodos. 2. Finalizar A con la lógica completa incluyendo la actualización de g(n) cuando se encuentra un camino mejor. 3. Implementar heurísticas adicionales:

	- Fichas mal ubicadas (contar cuántas fichas no están en su posición objetivo). 4. Permitir al usuario seleccionar la heurística a usar en tiempo de ejecución. Incluye la justificación de por qué Manhattan es admisible y consistente."*
Resultado / cómo se usó	Se completaron los algoritmos Greedy y A* con todas las funcionalidades. Se completó la implementación de las 2 heurísticas diferentes (Manhattan, fichas mal ubicadas). El sistema permitía seleccionar la heurística en tiempo de ejecución para comparar su impacto en el rendimiento.
Validación / ajuste del equipo	Se verificó que la heurística fichas mal ubicadas fuera admisible. Se probó que A* con diferentes heurísticas encontraba soluciones óptimas pero con diferente eficiencia.

N°	P05
Objetivo del prompt	Crear un sistema completo de comparación que ejecute todos los algoritmos (BFS, DFS, Greedy, A* con múltiples estados de prueba y genera estadísticas detalladas.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Ahora necesito un sistema de comparación completo para todos los algoritmos de búsqueda del 8-puzzle: 1. Clase Comparativa que: - Ejecute BFS, DFS, Greedy y A* con los mismos estados de prueba. - Mida nodos expandidos, costo de solución, tiempo de ejecución (ms). - Determine completitud y optimalidad de cada algoritmo. 2. Múltiples casos de prueba: - Fácil (2-3 movimientos). - Medio (10-15 movimientos). - Difícil (25-31 movimientos). 3. Generación de tabla comparativa en consola con formato. - Pasos realizados. - Nodos expandidos. - Nodos generados - Tiempo de ejecución (ms)"*
Resultado / cómo se usó	Se obtuvo una clase ResultadoBusqueda que guardaba los parámetros pedidos de todos los algoritmos y generaba tablas formateadas en la consola.
Validación / ajuste del equipo	Se probó con 10 estados diferentes de cada categoría

	(fácil, medio, difícil). Se verificó que BFS y A* siempre encontraban soluciones óptimas. Se ajustó el límite de tiempo para evitar ejecuciones infinitas en DFS.
--	---

N°	P06
Objetivo del prompt	Crear una interfaz de usuario interactiva completa en consola, sistema de configuración de casos de prueba, y generar la documentación final del proyecto.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para finalizar el proyecto del 8-puzzle modifica la interfaz de la consola en la clase main: 1. Crear una interfaz de usuario interactiva en consola que permita: - Ingresar estado inicial manualmente. - Seleccionar entre estados predefinidos (fácil, medio, difícil). - Elegir algoritmo(s) a ejecutar (individual o todos). - Ver solución paso a paso con animación del tablero. 2. Añadir validación de entrada (números 0-8, formato correcto). 3. Incluir el despliegue de las métricas calculadas en la consola. La interfaz debe ser intuitiva y guiar al usuario paso a paso."
Resultado / cómo se usó	Se obtuvo un programa completo con interfaz interactiva que permitía todas las funcionalidades solicitadas. La visualización paso a paso de la solución era clara y didáctica.
Validación / ajuste del equipo	Se probó la interfaz con usuarios externos para validar su usabilidad. Se ajustaron los mensajes de error y la validación de entrada. Se ajustó la clase main para incluir los casos de prueba y se incluye una opción que ejecutaba automáticamente todos los casos de prueba.

7. Referencias

- Iordan, Anca-Elena. 2016. "A Comparative Study of Three Heuristic Functions Used to Solve the 8-Puzzle". *Journal of Advances in Mathematics and Computer Science* 16 (1):1-18. <https://doi.org/10.9734/BJMCS/2016/24467>.
- GeeksforGeeks. (2025, 23 julio). How to check if an instance of 8 puzzle is solvable? GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/check-instance-8-puzzle-solvable/>
- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

https://api.pageplace.de/preview/DT0400.9781292401171_A41586057/preview-9781292401171_A41586057.pdf

- 8-Puzzle Programming assignment. (s. f.).
<https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/8puzzle/index.html>